

Lesson 10 Notes: Symmetric Matrix Storage and Retrieval

1. Main idea of Lesson 10

Lesson 10 teaches **symmetric matrices**.

Before this lesson, we studied these matrix types:

Lessons	Matrix type / idea	Main condition
Lessons 1-4	Diagonal matrix	<code>i == j</code>
Lessons 5-8	Lower triangular matrix	<code>i >= j</code>
Lesson 9	Upper triangular matrix	<code>i <= j</code>
Lesson 10	Symmetric matrix	<code>M[i][j] == M[j][i]</code>

The main idea of a symmetric matrix is:

The value above the main diagonal is the same as the matching value below the main diagonal.

So we do not need to store both halves of the matrix.

We can store only one triangular half and use the mirror property to get the other half.

2. Quick progress review

So far, the course has followed this path:

Lessons 1-4: Diagonal matrices

A diagonal matrix stores values only on the main diagonal.

Condition:

`i == j`

Example:

```
5 0 0 0
0 8 0 0
0 0 9 0
0 0 0 12
```

Stored array:

```
A = [5, 8, 9, 12]
```

Index formula:

```
index = i - 1
```

Lessons 5-8: Lower triangular matrices

A lower triangular matrix stores values on and below the main diagonal.

Condition:

```
i >= j
```

Example:

```
1 0 0 0
2 3 0 0
4 5 6 0
7 8 9 10
```

Stored row-major array:

```
A = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Lower triangular row-major formula:

```
index = i * (i - 1) / 2 + (j - 1)
```

Lesson 9: Upper triangular matrices

An upper triangular matrix stores values on and above the main diagonal.

Condition:

$$i \leq j$$

Example:

```
1 2 3 4
0 5 6 7
0 0 8 9
0 0 0 10
```

Lesson 9 taught that triangular matrices can be stored using either row-major or column-major mapping.

Lesson 10: Symmetric matrices

Lesson 10 uses the triangular matrix idea again.

But now the matrix is not zero on one side.

Instead, one side is a mirror copy of the other side.

That means:

$$M[i][j] == M[j][i]$$

Because of this, we can store only one half of the matrix.

3. What is a symmetric matrix?

A **symmetric matrix** is a square matrix where every value is equal to its mirror value across the main diagonal.

Example:

```
1 2 3 4
2 5 6 7
```

```
3 6 8 9
4 7 9 10
```

Look at the matching pairs:

```
M[1][2] = 2 and M[2][1] = 2
M[1][3] = 3 and M[3][1] = 3
M[1][4] = 4 and M[4][1] = 4
M[2][3] = 6 and M[3][2] = 6
M[2][4] = 7 and M[4][2] = 7
M[3][4] = 9 and M[4][3] = 9
```

Every value above the main diagonal matches a value below the main diagonal.

So the symmetric matrix rule is:

```
M[i][j] == M[j][i]
```

4. Main diagonal reminder

The **main diagonal** goes from the top-left corner to the bottom-right corner.

Example:

```
1 0 0 0
0 5 0 0
0 0 8 0
0 0 0 10
```

The main diagonal positions are:

```
M[1][1]
M[2][2]
M[3][3]
M[4][4]
```

For all main diagonal positions:

```
i == j
```

In a symmetric matrix, the main diagonal is stored only once.

The mirror idea mainly matters for positions where:

```
i != j
```

5. Mirror property

The most important property of a symmetric matrix is:

```
M[i][j] == M[j][i]
```

This means:

```
Row i, column j has the same value as row j, column i.
```

Example:

```
M[2][4] == M[4][2]
```

If:

```
M[4][2] = 7
```

then:

```
M[2][4] = 7
```

We do not need to store both values.

We can store one and calculate the other using the mirror position.

6. Why symmetric matrices save memory

A normal matrix stores every value.

For an $N \times N$ matrix, normal storage needs:

N^2 values

For example, a 4×4 normal matrix stores:

$4 \times 4 = 16$ values

But a symmetric matrix repeats values across the main diagonal.

Example:

```
1  2  3  4
2  5  6  7
3  6  8  9
4  7  9 10
```

The upper triangular part and lower triangular part contain matching values.

So we do not need to store both sides.

We can store either:

Lower triangular part

or:

Upper triangular part

Both choices are valid.

In this lesson, we use the **lower triangular row-major storage** method.

7. What part do we store?

For this lesson, we store the **lower triangular part** of the symmetric matrix.

Example full symmetric matrix:

```
1  2  3  4
2  5  6  7
3  6  8  9
4  7  9 10
```

The lower triangular part is:

```
1
2 5
3 6 8
4 7 9 10
```

This includes:

```
M[1][1]
M[2][1] M[2][2]
M[3][1] M[3][2] M[3][3]
M[4][1] M[4][2] M[4][3] M[4][4]
```

These positions follow the lower triangular condition:

```
i >= j
```

8. Stored array using lower triangular row-major order

Using this symmetric matrix:

```
1  2  3  4
2  5  6  7
3  6  8  9
4  7  9 10
```

We store only this part:

```
1
2 5
3 6 8
4 7 9 10
```

Row-major means we store row by row:

```
Row 1: 1
Row 2: 2, 5
Row 3: 3, 6, 8
Row 4: 4, 7, 9, 10
```

So the 1D array becomes:

```
A = [1, 2, 5, 3, 6, 8, 4, 7, 9, 10]
```

Mapping:

Matrix position	Array position
M[1][1]	A[0]
M[2][1]	A[1]
M[2][2]	A[2]
M[3][1]	A[3]
M[3][2]	A[4]
M[3][3]	A[5]
M[4][1]	A[6]
M[4][2]	A[7]
M[4][3]	A[8]
M[4][4]	A[9]

9. Storage size formula

A symmetric matrix stores one triangular half including the main diagonal.

The number of stored values is:

$$N(N + 1) / 2$$

For $N = 4$:

```
stored values = 4(4 + 1) / 2  
stored values = 4(5) / 2  
stored values = 20 / 2  
stored values = 10
```

So a 4×4 symmetric matrix can be stored using only 10 values instead of 16.

For $N = 5$:

```
stored values = 5(5 + 1) / 2  
stored values = 5(6) / 2  
stored values = 30 / 2  
stored values = 15
```

So a 5×5 symmetric matrix can be stored using only 15 values instead of 25.

10. Lower triangular row-major formula review

Since Lesson 10 stores the lower triangular part, it reuses the lower triangular row-major formula from Lesson 5.

Formula:

$$\text{index} = i * (i - 1) / 2 + (j - 1)$$

Use this formula directly when:

$$i \geq j$$

Meaning:

The requested position is already in the stored lower triangle.

The formula has two parts:

```
i * (i - 1) / 2
```

This skips all stored values in the previous rows.

```
j - 1
```

This moves inside the current row to the correct column.

So the formula means:

```
index = stored values before this row + movement inside this row
```

11. The new idea in Lesson 10: index swapping

In a lower triangular matrix, positions above the diagonal are structural zeros.

But in a symmetric matrix, positions above the diagonal are **not zero**.

They are mirror copies of positions below the diagonal.

So we cannot simply return 0 when:

```
i < j
```

Instead, we swap `i` and `j`.

Case 1: Position is in the lower triangle

If:

```
i >= j
```

Then the value is directly stored.

Use:

```
index = i * (i - 1) / 2 + (j - 1)
```

Case 2: Position is in the upper triangle

If:

$$i < j$$

Then the value is not directly stored.

But because the matrix is symmetric:

$$M[i][j] == M[j][i]$$

So we swap `i` and `j` and use the lower triangle position.

Use:

$$\text{index} = j * (j - 1) / 2 + (i - 1)$$

This is the main trick of Lesson 10.

12. Why index swapping works

Suppose someone asks for:

$$M[2][4]$$

This is above the main diagonal because:

$$2 < 4$$

Since we stored only the lower triangular part, `M[2][4]` is not directly stored.

But because the matrix is symmetric:

$$M[2][4] == M[4][2]$$

And `M[4][2]` is in the lower triangle because:

```
4 >= 2
```

So the program gets the value from `M[4][2]`.

That is index swapping.

Simple idea:

If the requested position is above the diagonal,
look at its mirror position below the diagonal.

13. `getSymmetric()` function

The `getSymmetric()` function returns a value from the symmetric matrix.

It uses lower triangular row-major storage.

```
int getSymmetric(int *A, int i, int j) {  
    if (i >= j) {  
        int index = (i * (i - 1) / 2) + (j - 1);  
        return A[index];  
    } else {  
        int index = (j * (j - 1) / 2) + (i - 1);  
        return A[index];  
    }  
}
```

14. Explanation of `getSymmetric()`

The function receives:

```
int *A
```

This is the 1D array that stores the lower triangular part.

```
int i
```

This is the row number.

```
int j
```

This is the column number.

The function checks:

```
if (i >= j)
```

If this is true, the position is already in the stored lower triangle.

So it uses:

```
int index = (i * (i - 1) / 2) + (j - 1);
```

Then it returns:

```
return A[index];
```

If `i >= j` is false, then:

```
i < j
```

So the position is in the upper triangle.

The function uses the mirror position by swapping `i` and `j`:

```
int index = (j * (j - 1) / 2) + (i - 1);
```

Then it returns:

```
return A[index];
```

15. Read `getSymmetric()` in simple words

The function means:

```
If the requested value is in the stored lower triangle:
    calculate its normal lower triangular index.
    return the stored value.
```

```
Otherwise:
    the requested value is in the upper triangle.
    swap row and column.
    calculate the mirror position index.
    return the stored value from the lower triangle.
```

16. Dry run 1: Get `M[4][2]`

We want:

```
M[4][2]
```

Values:

```
i = 4
j = 2
```

Step 1: Check whether the position is stored directly.

```
i >= j
4 >= 2
true
```

So `M[4][2]` is in the stored lower triangle.

Step 2: Use the lower triangular row-major formula.

```
index = i * (i - 1) / 2 + (j - 1)
```

Substitute values:

```
index = 4 * (4 - 1) / 2 + (2 - 1)
index = 4 * 3 / 2 + 1
index = 12 / 2 + 1
```

```
index = 6 + 1  
index = 7
```

So:

```
M[4][2] -> A[7]
```

Given the array:

```
A = [1, 2, 5, 3, 6, 8, 4, 7, 9, 10]
```

We have:

```
A[7] = 7
```

Final answer:

```
M[4][2] = 7
```

17. Dry run 2: Get `M[2][4]`

We want:

```
M[2][4]
```

Values:

```
i = 2  
j = 4
```

Step 1: Check whether the position is stored directly.

```
i >= j  
2 >= 4  
false
```

So `M[2][4]` is not in the stored lower triangle.

It is in the upper triangle.

Step 2: Use the symmetric property.

Because the matrix is symmetric:

```
M[2][4] == M[4][2]
```

So we swap `i` and `j`.

Original:

```
i = 2  
j = 4
```

Swapped for indexing:

```
row = j = 4  
column = i = 2
```

Step 3: Use the swapped formula.

```
index = j * (j - 1) / 2 + (i - 1)
```

Substitute values:

```
index = 4 * (4 - 1) / 2 + (2 - 1)  
index = 4 * 3 / 2 + 1  
index = 12 / 2 + 1  
index = 6 + 1  
index = 7
```

So:

```
M[2][4] -> mirror position M[4][2] -> A[7]
```

Given:

$A[7] = 7$

Final answer:

$M[2][4] = 7$

Important result:

$M[4][2]$ and $M[2][4]$ both return $A[7]$.

That is the symmetric matrix idea.

18. Dry run comparison: $M[4][2]$ and $M[2][4]$

Requested position	Directly stored?	Mirror used?	Array index	Value
$M[4][2]$	Yes	No	$A[7]$	7
$M[2][4]$	No	Yes, $M[4][2]$	$A[7]$	7

Both positions give the same answer because:

$M[2][4] == M[4][2]$

19. Set operation idea

The `set` operation stores a value in the symmetric matrix.

Since we store only the lower triangular part, `set` must also use the same mirror logic.

If the user sets a lower triangular position, store it normally.

If the user sets an upper triangular position, store it in the matching lower triangular mirror position.

20. `setSymmetric()` function

```
void setSymmetric(int *A, int i, int j, int x) {  
    if (i >= j) {  
        int index = (i * (i - 1) / 2) + (j - 1);  
        A[index] = x;  
    } else {  
        int index = (j * (j - 1) / 2) + (i - 1);  
        A[index] = x;  
    }  
}
```

21. Explanation of `setSymmetric()`

The function receives:

```
int *A
```

The 1D array storing the lower triangular part.

```
int i
```

The row number.

```
int j
```

The column number.

```
int x
```

The value to store.

If:

```
i >= j
```

then the position is in the lower triangle, so store normally:

```
A[index] = x;
```

If:

```
i < j
```

then the position is in the upper triangle.

Instead of storing a new duplicate value, store `x` in the lower triangle mirror position.

22. Dry run of `setSymmetric(A, 2, 4, 7)`

Call:

```
setSymmetric(A, 2, 4, 7);
```

Values:

```
i = 2  
j = 4  
x = 7
```

Step 1: Check condition.

```
i >= j  
2 >= 4  
false
```

So the requested position is in the upper triangle.

Step 2: Use mirror position.

```
M[2][4] mirrors M[4][2]
```

Step 3: Calculate index using swapped formula.

```
index = j * (j - 1) / 2 + (i - 1)  
index = 4 * (4 - 1) / 2 + (2 - 1)
```

```
index = 4 * 3 / 2 + 1
index = 6 + 1
index = 7
```

Step 4: Store the value.

```
A[7] = 7
```

So even though the user gave `M[2][4]`, the program stores the value at the lower triangular mirror position `M[4][2]`.

23. Display operation idea

The display operation prints the full symmetric matrix.

The stored array contains only one triangular half.

But the output should show the full square matrix.

So display loops through every matrix position:

```
M[1][1], M[1][2], M[1][3], ...
M[2][1], M[2][2], M[2][3], ...
...
```

For each position, it calls `getSymmetric()`.

This is useful because `getSymmetric()` already knows whether to:

```
use the stored position directly
```

or:

```
use the mirror position
```

24. `displaySymmetric()` function

```
void displaySymmetric(int *A, int n) {  
    for (int i = 1; i <= n; i++) {  
        for (int j = 1; j <= n; j++) {  
            cout << getSymmetric(A, i, j) << " ";  
        }  
        cout << endl;  
    }  
}
```

25. Explanation of `displaySymmetric()`

The function uses two loops.

Outer loop:

```
for (int i = 1; i <= n; i++)
```

This controls the rows.

Inner loop:

```
for (int j = 1; j <= n; j++)
```

This controls the columns.

For each position, it prints:

```
getSymmetric(A, i, j)
```

That means display does not need to manually check whether the position is in the lower or upper triangle.

The `getSymmetric()` function handles that logic.

26. Full example using the stored array

Suppose we store:

```
A = [1, 2, 5, 3, 6, 8, 4, 7, 9, 10]
```

This represents the lower triangular part:

```
1
2 5
3 6 8
4 7 9 10
```

The full symmetric matrix is:

```
1 2 3 4
2 5 6 7
3 6 8 9
4 7 9 10
```

How?

The lower part is stored directly:

```
M[4][2] = 7
```

The upper part is found by mirror lookup:

```
M[2][4] = M[4][2] = 7
```

27. Full beginner-friendly C++ example

```
#include <iostream>
using namespace std;

int getSymmetric(int *A, int i, int j) {
    if (i >= j) {
        int index = (i * (i - 1) / 2) + (j - 1);
        return A[index];
    } else {
        int index = (j * (j - 1) / 2) + (i - 1);
        return A[index];
    }
}
```

```

}

void setSymmetric(int *A, int i, int j, int x) {
    if (i >= j) {
        int index = (i * (i - 1) / 2) + (j - 1);
        A[index] = x;
    } else {
        int index = (j * (j - 1) / 2) + (i - 1);
        A[index] = x;
    }
}

void displaySymmetric(int *A, int n) {
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            cout << getSymmetric(A, i, j) << " ";
        }
        cout << endl;
    }
}

int main() {
    int n = 4;
    int size = n * (n + 1) / 2;
    int *A = new int[size]{};

    setSymmetric(A, 1, 1, 1);
    setSymmetric(A, 2, 1, 2);
    setSymmetric(A, 2, 2, 5);
    setSymmetric(A, 3, 1, 3);
    setSymmetric(A, 3, 2, 6);
    setSymmetric(A, 3, 3, 8);
    setSymmetric(A, 4, 1, 4);
    setSymmetric(A, 4, 2, 7);
    setSymmetric(A, 4, 3, 9);
    setSymmetric(A, 4, 4, 10);

    cout << "Value at M[4][2]: " << getSymmetric(A, 4, 2) << endl;
    cout << "Value at M[2][4]: " << getSymmetric(A, 2, 4) << endl;

    cout << "\nFull symmetric matrix:" << endl;
    displaySymmetric(A, n);

    delete[] A;

    return 0;
}

```

28. Expected output

```
Value at M[4][2]: 7
Value at M[2][4]: 7

Full symmetric matrix:
1 2 3 4
2 5 6 7
3 6 8 9
4 7 9 10
```

The important result is:

```
M[4][2] and M[2][4] both return 7.
```

That happens because they are mirror positions.

29. Upper triangular storage is also possible

This lesson uses lower triangular storage.

But symmetric matrices can also be stored using the upper triangular part.

If we choose upper triangular storage, we store:

```
1 2 3 4
  5 6 7
    8 9
      10
```

The stored array would be different because the storage order is different.

But the idea is the same:

```
Store one half.
Use the mirror property for the other half.
```

For this lesson, we focus on lower triangular row-major storage because we already know the lower triangular formula.

30. Difference between lower triangular matrix and symmetric matrix

A lower triangular matrix and a symmetric matrix may both store the lower triangular part, but they are not the same concept.

Matrix type	What happens above the diagonal?
Lower triangular matrix	Values above the diagonal are structural zeros.
Symmetric matrix	Values above the diagonal are mirror copies of values below the diagonal.

Example lower triangular matrix:

```
1 0 0 0
2 5 0 0
3 6 8 0
4 7 9 10
```

Here:

```
M[2][4] = 0
```

Example symmetric matrix:

```
1 2 3 4
2 5 6 7
3 6 8 9
4 7 9 10
```

Here:

```
M[2][4] = M[4][2] = 7
```

So for lower triangular matrices, `i < j` means return `0`.

For symmetric matrices, `i < j` means swap the indexes and return the mirror value.

31. Common mistake

A common mistake is treating the upper half of a symmetric matrix as zero.

That is wrong.

In a symmetric matrix:

```
M[2][4] is not automatically 0.
```

Instead:

```
M[2][4] = M[4][2]
```

Only triangular matrices have structural zeros on one side.

Symmetric matrices have mirrored values.

32. Another common mistake

Another common mistake is storing both `M[i][j]` and `M[j][i]` separately.

Example:

```
Store M[2][4]
Store M[4][2]
```

This wastes memory because both values must be the same.

Better approach:

```
Store only one of them.
Use the mirror property to get the other.
```

33. Beginner mental model

Think of a symmetric matrix like a mirror.

The main diagonal is the mirror line.

Upper half is reflected from the lower half.

So if you know the lower half, you already know the upper half.

Example:

Asked: $M[2][4]$
Mirror: $M[4][2]$
Return: value stored for $M[4][2]$

The computer does not need to store both values.

34. Memory complexity

A symmetric matrix stores:

$N(N + 1) / 2$ values

This is about half of a full matrix.

In Big-O notation:

Storage = $O(N^2)$

This may seem strange at first because we are saving memory.

But Big-O ignores constant factors.

A full matrix stores:

N^2 values

A symmetric matrix stores:

$N(N + 1) / 2$ values

This is roughly:

$$N^2 / 2$$

Even though it is half, it still grows like N^2 .

So the Big-O space complexity is:

$$O(N^2)$$

35. Time complexity of `getSymmetric()`

The `getSymmetric()` function takes constant time.

It does only:

```
one condition check  
one formula calculation  
one array access
```

So:

```
getSymmetric() time = O(1)
```

This is very efficient because the function does not use loops.

36. Time complexity of `setSymmetric()`

The `setSymmetric()` function also takes constant time.

It does only:

```
one condition check  
one formula calculation  
one assignment
```

So:

```
setSymmetric() time =  $O(1)$ 
```

37. Time complexity of display

The display function must print the full visible matrix.

For an $N \times N$ matrix, there are:

N^2 visible positions

So display uses nested loops.

Therefore:

```
displaySymmetric() time =  $O(N^2)$ 
```

Even though storage is reduced, displaying the whole matrix still takes $O(N^2)$ time.

38. Complexity table

Operation	Time complexity	Space complexity	Reason
Storage	—	$O(N^2)$	Stores $N(N + 1) / 2$ values, about half of full matrix storage.
<code>setSymmetric()</code>	$O(1)$	—	One condition, one formula, one assignment.
<code>getSymmetric()</code>	$O(1)$	—	One condition, one formula, one array lookup.
<code>displaySymmetric()</code>	$O(N^2)$	—	Must print every visible matrix position.

39. Key formulas

Stored values

$\text{stored values} = N(N + 1) / 2$

Direct lower triangular position

Use when:

$i \geq j$

Formula:

$\text{index} = i * (i - 1) / 2 + (j - 1)$

Mirrored upper triangular position

Use when:

$i < j$

Formula:

$\text{index} = j * (j - 1) / 2 + (i - 1)$

Symmetric property

$M[i][j] == M[j][i]$

40. Main takeaway

The main idea of Lesson 10 is:

A symmetric matrix does not need both halves stored.

Because:

```
M[i][j] == M[j][i]
```

we can store only one triangular half.

In this lesson, we store the lower triangular half using row-major storage.

When the requested position is already in the lower triangle:

```
i >= j
```

we use the normal lower triangular formula:

```
index = i * (i - 1) / 2 + (j - 1)
```

When the requested position is in the upper triangle:

```
i < j
```

we swap `i` and `j`:

```
index = j * (j - 1) / 2 + (i - 1)
```

That index swapping is the heart of symmetric matrix storage.

41. Final beginner summary

A symmetric matrix is like a mirror.

The main diagonal is the mirror line.

Values above the diagonal match values below the diagonal.

So the computer only stores one side.

If a value is requested from the stored side, the program uses it directly.

If a value is requested from the other side, the program swaps the row and column and returns the mirror value.

The most important rule is:

```
M[i][j] == M[j][i]
```

The most important programming trick is:

```
If i < j, swap i and j before calculating the lower triangular index.
```

That is Lesson 10.